DOMTAK

Technical Handbook Version 1 (V1)

Everything the frontend and backend teams need to implement, deploy and operate Domtak on Android, iOS and the web.

Android · iOS · Web

Music social network · events and tickets · artist, venue and lesson booking · Berlin

Document
Version 1.0
24 October 2026

Code
github.com/Nasseralmagout/DOMTAK-V1
branch main

Audience
Frontend, backend, DevOps, QA and product

Contents

About this document 4
How to read the labels 4
Sources this document is based on 4
1. Project overview 5
1.1 Product summary 5
1.2 Current status 5
1.3 Scope of work for the teams 6
2. Product rules, roles and permissions 7
2.1 Accounts and workspaces 7
2.2 Permission matrix 7
2.3 Product owner rules 8
2.4 Blueprint rules that the backend must guarantee 8
3. Screens and navigation 10
3.1 Navigation structure 10
3.2 Screen inventory 10
4. Front-end architecture (Android, iOS, Web) 14
4.1 Technology stack 14
4.2 Folder structure 15
4.3 How data flows today 15
4.4 Design system 15
4.5 Platform specifics 16
4.6 From demo to production: integration plan 16
4.7 Native modules to add 17
4.8 Working on the app 17
5. Backend architecture 18
5.1 Recommended stack 18
5.2 System overview 18
5.3 Modules 19
5.4 Authentication and authorization 19
5.5 Background jobs and timers 20
5.6 Realtime, notifications and messaging 20
5.7 Media 21
5.8 Search and location 21
6. Data model 22
6.1 Accounts, access and social 22
6.2 Providers, events and tickets 23

6.3 Bookings, payments and support 24
6.4 Booking lifecycle 25
6.5 Payment states 26
6.6 Other lifecycles 27
6.7 Pricing and scoring rules 27

7. API specification (REST v1) 28
7.1 Conventions 28
7.2 Auth and account 28
7.3 Professional access 29
7.4 Social 29
7.5 Messages and notifications 30
7.6 Discovery, providers and reviews 30
7.7 Events, invitations and tickets 31
7.8 Bookings (customer side) 31
7.9 Provider workspace 32
7.10 Payments, subscriptions, media and support 32
7.11 Admin (staff permissions) 33

8. Payments 34
8.1 Recommended model 34
8.2 Listed-price booking 34
8.3 Quotes, tickets and contributions 35
8.4 Cancellations, refunds and disputes 35
8.5 Fees, commission and payouts 35
8.6 Pro subscription 35

9. External services to activate 37

10. Environments, deployment and release 39
10.1 Environments 39
10.2 Configuration 39
10.3 Backend deployment 40
10.4 Web app 40
10.5 Android release 41
10.6 iOS release 41
10.7 Build profiles and over-the-air updates 41
10.8 CI/CD pipelines (GitHub Actions) 42
10.9 Go-live checklist 42

11. Security, privacy and compliance 43
11.1 Security measures 43
11.2 Privacy (GDPR) 43
11.3 Legal texts and platform rules to review 44

12. Testing and acceptance 45
12.1 Test strategy 45
12.2 Acceptance criteria (from the blueprint) 45
12.3 Flow coverage 46
13. Open decisions 47
14. Suggested work plan 48
Appendix A. Glossary 49
Appendix B. Demo-only items to remove or replace 50
Appendix C. Quick reference 51

About this document

Domtak V1 exists today as a complete, working front-end: one Expo / React Native codebase that runs as an Android app, an iOS app and a web app. Every screen and flow works, but on sample data stored on the device. There is no server yet, and payments, messages and notifications are simulated. This handbook describes what is built, what the backend must provide, which external services must be activated, and how to ship all three platforms.

How to read the labels

Label	Meaning
BUILT	Exists in the V1 front-end today (UI, navigation, client-side rules), running on demo data.
TO BUILD	Must be implemented: backend services, real integrations, or production hardening of the front-end.
DECISION	An open product, legal or business decision from the blueprint. Do not hard-code an answer; build it configurable or wait for the decision.
LATER	Outside V1 scope (blueprint flows F28–F30). Keep the data model open to it, but do not build it now.

Recommendations for technologies and vendors are marked as such. They are sensible defaults that fit the existing code and EU/GDPR hosting; the teams may change them, provided the contracts in this document (data model, API behaviour, state machines and acceptance criteria) are kept.

Sources this document is based on

Source	What it contains
Code repository	`github.com/Nasseralmagout/DOMTAK-V1`, branch `main`. The app is in `mobile/`. The same history also lives in `Nasseralmagout/domtak` (pull request #1).
Product blueprint	"Domtak User Flows and Dashboards" (1 October 2026): 16 sections and flows F01–F30, the booking and payment state tables, dashboards and the list of open decisions.
Product owner rules	Rules added on top of the blueprint (section 2.3): organizer-only public events, member photos only in comments, clearer venue page with parking and garden separate, "would book again" rating for musicians, nearby-events map with filters, catering and security priced by attendance.
Design canvas	"Domtak Android App" design artifact, page "V1 · Wireframe & final": final designs of all 136 V1 screens (private; the owner shares it from the page's Share menu).
Web preview	Vercel project built from `vercel.json` in the repository (Root Directory `mobile` or empty, framework preset "Other").

1. Project overview

1.1 Product summary

Domtak is a music-first platform for Berlin that connects fans, musicians (solo artists, bands and DJs), venues and event organizers. It combines four products in one app:

- **Social:** an Instagram-style home feed (For You, Following, Nearby), stories, live, comments, follows, friends, direct messages and platform-run music communities.
- **Events and tickets:** discovery by category, filters and a nearby-events map, event pages and discussions, ticket checkout, an e-ticket wallet with QR codes, and door check-in.
- **Booking:** booking musicians, venues and music lessons, at a listed price or by itemised quote, with a 24-hour provider response window, a payment state for every booking, change requests, cancellations, disputes and reviews.
- **Professional workspaces:** reviewed applications for artists, venues and organizers; requests, availability, services and pricing, earnings and payouts, event management, promotion, guest lists and a QR scanner; plus an admin console for applications, booking exceptions, finance and moderation.

1.2 Current status

Area	Status	Details
Screens and navigation	BUILT	90 route files (136 designed screen states) covering flows F01–F27 of the blueprint. Same code on Android, iOS and web.
Business rules (client side)	BUILT	Booking and payment state machine, 24 h deadline and 12 h reminder, quotes, pricing (catering, security, travel, equipment, fees), ratings, event lifecycle, permissions. These are in `mobile/src/lib` and `mobile/src/store` and must be re-implemented on the server as the source of truth.
Data	TO BUILD	All content is sample data (`src/data/mock.ts`) and user actions are stored on the device (zustand + AsyncStorage, key `domtak-app`). Needs the backend API.
Accounts and sign-in	TO BUILD	UI is complete (register, verify code, sign in, forgot password, two-step verification, change email/phone, devices, data export, delete account). No real authentication yet.
Payments	TO BUILD DECISION	Checkout, failure, retry, refund and payout screens exist; nothing is charged. Payment provider and authorization model are open decisions.
Messages, notifications, push	TO BUILD	Inbox, chat and notification screens exist on local data. No realtime transport, push or email.
Map and location	TO BUILD	Nearby map with filters works on a static Berlin image with positioned pins. Needs a map SDK, device location and geo queries.
Media upload	TO BUILD	Post, story and comment-photo screens use bundled sample images. Needs image picking, upload and a CDN.
QR scanner	BUILT (camera)	Uses `expo-camera` . Validation runs against local data; must call the server.
Admin console	BUILT (demo)	Applications, bookings, finance, reports. Opened by a demo switch; needs staff authentication and role-based access.

Area	Status	Details
Builds	BUILT	Android bundle and web export build cleanly. `eas.json` has preview (APK) and production (AAB) profiles. iOS builds need an Apple Developer account.

1.3 Scope of work for the teams

Backend team

- API and database for every entity in section 6
- Authentication, roles, applications and staff access
- Booking, quote and payment state machines with timers
- Payment provider integration, webhooks, refunds, payouts
- Tickets, QR validation and check-in
- Feed, comments, messages (realtime), notifications
- Media storage, search, geo queries
- Admin, moderation, support cases, audit log
- Hosting, monitoring, backups, GDPR tooling

Frontend team

- API client layer; replace mock data and local actions
- Secure token storage and session handling
- Native modules: push, location, maps, image picker, payments
- Remove demo controls; real dates and real admin access
- Error, loading and offline states against a real network
- Replace watermarked sample photos and the map image
- Store builds, signing, submission, over-the-air updates
- Web hosting and domain
- Automated tests for the acceptance criteria (section 12)

2. Product rules, roles and permissions

The rules below are product requirements. The front-end already follows them. The backend must enforce every one of them independently: a client can be modified, so no rule may depend on the app hiding a button.

2.1 Accounts and workspaces

- Every account starts as a **Member** (internal role `fan`). The member workspace is always available.
- Three **professional roles** need a reviewed application: **Artist** (`musician`: solo artist, band or DJ), **Venue** (`venue`) and **Organizer** (`organizer`). Application states are `pending`, `info` (more information requested, with the exact missing items), `approved` and `rejected` (with a reason). Submitting an application never grants access by itself.
- One account can hold several approved workspaces (`approvedRoles`). Exactly one is **active** at a time (`role`). Switching to a role that is not approved is refused.
- **Staff** (admin, reviewer, support, finance, moderator) are not app roles. They are separate staff permissions on top of an account and are checked on every admin endpoint (section 5.4). DECISION The reviewer, support and finance authority hierarchy is open (blueprint F03, F24–F27).

2.2 Permission matrix

"Approved" means the role is in `approvedRoles` and is the active workspace. The server checks both role and ownership (for example, only the provider of a request may answer it).

Action	Member	Artist	Venue	Organizer	Staff
Browse feed, events, artists, venues, lessons, map	Yes	Yes	Yes	Yes	Yes
Like, comment, follow, friends, direct messages, join communities	Yes	Yes	Yes	Yes	—
Attach photos to comments	Yes	Yes	Yes	Yes	—
Publish feed posts and stories	No	Approved	Approved	Approved	—
Create private events (invite guests, cost sharing)	Yes	Yes	Yes	Yes	—
Create and publish public events, sell public tickets	No (saved as draft)	No	No	Approved	—
Promote an event (paid placement)	No	No	No	Approved, own events	—
Buy tickets; book artists, venues and lessons	Yes	Yes	Yes	Yes	—
Receive booking requests, send quotes, counter offers	No	Approved	Approved	No	Read only
Availability, services and pricing, lessons	No	Approved	Availability	No	—
Venue listing (parking, garden, catering, security, charges)	No	No	Approved	No	—
Earnings and payouts	No	Approved	Approved	Approved	Finance

Action	Member	Artist	Venue	Organizer	Staff
Ticket scanning and guest list	Host or organizer of that event, plus staff they delegate (artists and venues get no automatic access)				—
Apply for a professional role	Yes	Yes, for another role			—
Review applications, investigate bookings, finance queue, moderation, support replies	No	No	No	No	By staff permission

2.3 Product owner rules

Rule	Requirement
Public events	Only approved organizers create and publish public events. Members, artists and venues create private events only. A public event saved by someone who cannot publish stays a draft; `setEventState(published)` is refused.
Member photos	Members can upload photos only inside comments (post comments and event discussions). Feed posts and stories are for approved professional accounts.
Venue page	The venue page opens with key facts (capacity, rooms and stages, price or "Request a quote", rating or "New"). Parking and garden/outdoor space are separate items, each with its own details (parking: available, kind, spaces, notes; garden: available, capacity, covered, notes).
Musician rating	Musicians are not rated with stars. Their score is the share of verified reviews (from completed bookings) that answered "Would book again" = yes, shown with the count, plus praise tag counts: On time, Great sound, Read the crowd, Professional, Good value, Easy to work with. With no verified reviews: "No verified bookings yet". Venues and events keep 1–5 stars.
Nearby map	A map of nearby events with filters: distance (1, 2, 5, 10 km or any), today, this weekend, free, live music, genre, plus the general filters (category, date, price, area, sort). Optional venue pins.
Catering and security	Venues can offer catering priced per guest (`pricePerGuest × max(guests, minGuests)`) and security priced by attendance (`ceil(guests / guestsPerGuard) × pricePerGuard`). Security can be mandatory from a guest count (`requiredFrom`).

2.4 Blueprint rules that the backend must guarantee

1. A member cannot publish a public event without organizer approval.
2. A professional application status is visible and does not grant access before approval.
3. Every provider booking has its own response deadline and its own financial record. Artist and venue bookings for the same event are independent.
4. The provider has 24 hours to respond from the moment a valid request reaches them. A reminder goes out after 12 hours, only while the request is still awaiting a response.
5. An unanswered request expires at 24 hours. Expiry starts the release or refund; it never claims the release is already complete.
6. Provider acceptance alone does not confirm a booking. A booking is confirmed only when the payment is verified.
7. Declining one provider request does not cancel other confirmed bookings of the same event. Cancelling an event flags linked bookings for the cancellation process; it does not cancel them automatically.

8. Agreed terms (price lines, date, quote version) are stored on the booking and stay unchanged when the provider later changes their prices.
9. A payment retry happens on the same booking. It never creates a second booking or a second charge.
10. Ticket validation refuses wrong-event, cancelled, refunded and already-used tickets, and shows when an already-used ticket was checked in.
11. A private-event guest ticket (cost sharing) is issued only after the contribution payment succeeds. A failed payment never issues a ticket.
12. Support cannot promise a refund before an authorized decision. A support case cannot be closed while a linked refund or release is still pending.
13. Dashboard actions respect ownership and role permissions. Financial totals separate booking value, fees, refunds and provider payouts.
14. Empty states show the real absence of activity. No fabricated numbers or results.
15. Communities are created by the platform only in V1. Blocking someone also ends the friendship and declines pending friend requests. A reported account never learns who reported it.

3. Screens and navigation

3.1 Navigation structure

Bottom tabs
Home · Artists · Create (+) · Events · Calendar

Header (Home)
Menu · Messages · Map · Notifications

Side menu
Profile, Dashboard, Switch workspace, Pro tools, My events, Events near me, Communities, Friends, Music lessons, Bookings, Tickets, Wishlist, Payment methods, Notifications, Pro plan, Help, Support cases, Admin (staff), Settings, Log out

Routing uses Expo Router (file-based). Every file in `mobile/src/app` is a screen; `_layout.tsx` files define navigators. The URL path of a screen is the same on all three platforms, so deep links and web URLs share one scheme: `domtak://event/bunker7` on mobile and `https://<web-domain>/event/bunker7` on the web. Signed-out users are redirected to `/login?next=<path>` and return to the path after signing in.

3.2 Screen inventory

90 route files implement the 136 screen states on the design canvas. “Data” lists what the screen needs from the API (section 7).

Onboarding, sign-in and access

Route	Screen	Data and actions
`/`	Entry redirect	Session check → onboarding, login or Home
`/onboarding`	Onboarding (3 slides)	Static
`/login`, `/register`	Sign in, register	Auth: sign-in, register (member), remember me
`/verify`	Verify email code	Auth: verify 6-digit code, resend
`/forgot-password`	Reset password (3 steps)	Auth: request code, verify, set new password
`/role`, `/profile-setup`	Choose role, profile setup / edit	Profile update, avatar upload, genres; professional roles route to the application
`/apply`	Professional application form	Role-specific fields (stage name, genres, links, photos, address, capacity…); submit application
`/application` (`/pro-verification`)	Application status	Status, missing items, reason, history; resubmit
`/workspace`	Switch workspace	Approved roles; switch active workspace

Home and social

Route	Screen	Data and actions
`/(tabs)` (`?tab=forYou|following|nearby`)	Home feed	Feed pages, stories row, attention banner (action queue), quick links, communities strip, upcoming events, sponsored slot

Route	Screen	Data and actions
`/story/[id]`, `/live/[id]`	Story viewer, live	Story frames; live stream (see section 9, video) with "Book this artist"
`/post/[id]`	Post and comments	Post, comments with photos, like, reply, report, hide removed content
`/new-post` (`?story=1`)	New post or story	Professionals only (members see an explanation); media upload, caption, visibility Public/Followers
`/create-menu`, `/(tabs)/create`	Create (+) sheet	Options per workspace and permission
`/menu`	Side menu	Profile summary, workspace, counts
`/notifications`	Notifications	Transaction notices (essential) and social updates (optional)
`/search`	Search	Events, artists, lessons, venues, communities, organizers, people
`/communities`, `/community/[id]`	Communities	Platform communities, join/leave, rules, posts, events
`/friends` (`?tab=requests`)	Friends and requests	Send, accept, decline, cancel, remove
`/profile/[id]`, `/connections/[id]`	Profile, followers / following	Profile, posts grid, follow, message, block, report
`/inbox`, `/chat/[id]`	Messages	Conversations (all / booking chats), messages (text, image, voice), read state

Events and tickets

Route	Screen	Data and actions
`/(tabs)/events`, `/events/[category]`	Events tab, event lists	Discoverable events by category, filters, sort
`/filters`	Filters	Category, date, price, genre, area, distance, sort
`/map`	Nearby events map	Events near a location with filters; optional venue pins
`/event/[id]`, `/event-comments/[id]`	Event page, discussion	Event, line-up, venue, tickets, going, discussion with photos
`/tickets/[eventId]`, `/checkout`, `/success`	Choose tickets, checkout, success	Ticket types and stock; payment; issued tickets
`/ticket/[id]`	E-ticket	QR code, status (valid, used with time, cancelled, refunded), help
`/review/[id]`	Leave a review	Stars (events, venues) or "Would book again" + praise tags (musicians)
`/create-event` (`?private=1`, `?edit=id`)	Create / edit event (4 steps)	Details, location, tickets, preview; draft, publish (organizer), edit with sold-ticket protection
`/event-created`	Event created	Share, manage, edit
`/my-events`, `/event-workspace/[id]`	My events, event workspace	State, readiness, budget lines, providers, guests, cost sharing, cancel event
`/invite`	Invite guests	Friends and followers to invite

Route	Screen	Data and actions
`/invitation/[id]`	Invitation (guest)	RSVP, contribution payment, ticket

Artists, venues and lessons

Route	Screen	Data and actions
`/(tabs)/artists`, `/artists/[kind]`	Artists tab, DJs / musicians / bands	Artist lists, filters, “would book again” sort
`/artist/[id]`, `/ratings/[id]`	Artist profile, reviews	Profile, score, praise tags, lessons, charges, availability, book / request quote
`/venues`, `/venue/[id]`	Venues, venue page	Key facts, parking, garden, facilities, requirements, catering, security, charges, live estimate, stars
`/lessons`, `/lesson/[id]`	Lessons	Lesson search (instrument, format, level), lesson details, book
`/wishlist`	Wishlist	Saved events, artists, venues

Booking and payments

Route	Screen	Data and actions
`/booking/new`	Booking request	Artist, venue or lesson; date, hours, guests, extras; price lines; listed request or quote request; event link
`/booking/[id]`	Booking status	Lifecycle, deadline, payment state, actions per status (pay, retry, accept quote, change, cancel, help, review)
`/quote/[id]`	Quote	Itemised quote, validity, accept or decline
`/booking/issue`	Cancel, refund, dispute	Reason, outcome, evidence, support case
`/my-bookings` (`?tab=`)	Your bookings and tickets	Musicians, venues, lessons, tickets
`/payment-methods`	Payment methods	Saved methods, add card, default
`/subscription`	Pro plan	€9,99 / month or €89 / year, first month free

Calendar, account and support

Route	Screen	Data and actions
`/(tabs)/calendar`	Calendar	Month, week, overview of tickets, bookings, events, availability
`/dashboard`	Member / host dashboard	Action queue, upcoming, bookings, events
`/settings`	Settings	Workspace, notifications, privacy, language, admin console (staff)
`/account-security`	Account and security	Change email/phone (code), password, two-step verification, devices, data export, delete account
`/blocked`	Blocked users	List, unblock
`/help`	Help and FAQ	Static content, contact
`/support`, `/support/new`, `/support/[id]`	Support cases	Cases with reference, linked record, timeline, reply, close, appeal
`/legal` (`?tab=`)	Terms, privacy, imprint	Static legal text (to be replaced by counsel’s final text)

Professional tools

Route	Screen	Data and actions
`/pro/workspace`	Workspace home (per role)	Requests due, readiness checklist, earnings summary, insights
`/pro/requests`, `/pro/request/[id]`	Incoming requests	Accept, decline (reason), propose changes / revised quote, conflict check, deadline
`/pro/availability`	Availability	Weekdays, blocked dates, fee, minimum hours, radius, instant book
`/pro/services`	Services and pricing	Listed or quote, travel and equipment charges, lessons
`/pro/earnings`	Earnings and payouts	Ledger, commission, payouts, report an issue
`/pro/events`, `/pro/promote/[eventId]`	Organizer events, promote	Events, sales, edit; promotion budget, days, radius, audience
`/pro/attendees/[eventId]`, `/pro/scan`	Guest list, QR scanner	Attendees, check-in state; scan and validate tickets
`/pro/venue-listing`	Venue listing editor	All venue fields incl. parking, garden, catering, security, extra charges; completion checklist

Admin console (staff)

Route	Screen	Data and actions
`/admin`	Overview	Metrics, queues, booking exceptions
`/admin/applications`	Applications	Review queue; approve, request information, reject (with reason)
`/admin/bookings` (`?request=`)	Booking investigation	Search, timeline, participants, payment state, exceptions
`/admin/finance`	Finance queue	Pending captures, releases, refunds, payouts, totals
`/admin/reports`	Moderation	Reports; dismiss, restrict, remove (audit logged)

4. Front-end architecture (Android, iOS, Web)

One TypeScript codebase in `mobile/` produces all three apps. Android and iOS are native apps built with Expo (React Native); the web app is the same code rendered by React Native Web and exported as a static single-page app.

4.1 Technology stack

Concern	Technology (version in repo)	Notes
Framework	Expo SDK 57 (`expo ~57.0.28`)	Continuous Native Generation: there are no committed `android/` or `ios/` folders. Native config lives in `app.json` and config plugins.
UI runtime	React Native 0.86.3, React 19.2.3	React Native Web 0.21 for the web build.
Language	TypeScript ~6.0 (strict)	`npm run typecheck`
Navigation	Expo Router ~57 (typed routes)	Routes in `src/app` ; URL = deep link path.
State	zustand 5 + persist (AsyncStorage)	Today it holds all demo data. In production, keep it for session and UI state only (section 4.6).
Images	expo-image	Bundled sample images today (`src/data/images.ts` , keys of type `ImgKey`).
Camera / QR	expo-camera, react-native-qrcode-svg	Ticket scanner and e-ticket QR codes.
Fonts and icons	Poppins, DM Sans, Lilita One (wordmark); Material Community Icons	Web uses a generated SVG path map: `npm run icons` → `src/components/iconPaths.ts` .
Other Expo modules	expo-linear-gradient, expo-haptics, expo-clipboard, expo-linking, expo-splash-screen, expo-status-bar, expo-font	
Lint	ESLint 9 with eslint-config-expo	`npm run lint` (CI uses `CI=1 npx eslint .`)
Builds	EAS Build / Submit / Update; Vercel for web	`eas.json` , `vercel.json`

4.2 Folder structure

```
mobile/
├─ app.json            Expo config: name, bundle ids (de.domtak.app), scheme (domtak), plugins,
permissions
├─ eas.json            EAS build profiles: preview (Android APK), production (Android AAB, auto-
increment)
├─ vercel.json         Web build on Vercel (also one at the repo root)
├─ assets/             Icons, splash, illustrations, sample photos (98 images)
├─ scripts/            gen-icon-paths.mjs; svg-export/ and design-export/ (design tooling, not part
of the app)
└─ src/
   ├─ app/             90 route files (screens) and layouts
   ├─ components/      Shared UI: ui.tsx (Button, T, Pill, Chip, Toggle, Icon…), layout.tsx (Screen,
headers,
   │                   PermissionNotice, DemoBar), cards.tsx, PostCard.tsx, ActionSheet.tsx,
TabBar.tsx…
   ├─ data/            types.ts (domain model), mock.ts (sample data), images.ts (image registry)
   ├─ lib/             Business rules: booking, pricing, ratings, events, access, support, admin,
dashboards,
   │                   promotions, format (dates, euro), eventHost
   ├─ store/           useApp.ts (main store and actions), filters, social, discovery, services,
toast
   └─ theme/           Colors, fonts, radius, shadows
```

4.3 How data flows today

- **Sample data** (`src/data/mock.ts`): profiles, venues, events, posts, stories, bookings b1–b7, requests r1–r6, lessons, communities, support case DT-4821, admin applications.
- **Store** (`src/store/useApp.ts`): one zustand store persisted to AsyncStorage (web: localStorage) under the key `domtak-app` . About 90 actions implement every user action (section 7 maps them to endpoints). Selectors must return stable values (zustand v5).
- **Rules** (`src/lib/*`): pure functions for labels, state transitions, prices, deadlines, readiness and permissions. Keep them for display on the client, and re-implement the authoritative versions on the server.
- **Demo clock:** `TODAY = 2026-10-14T12:00` in `src/lib/format.ts` keeps the sample data current. Replace it with the real time in production.

4.4 Design system

Token	Value and use
Brand colors	Deep purple `#2E0B5E` (headers, splash), brand `#3B1F8B` (actions), gold `#F2B632` (highlights), lilac `#C9B8F0` , background `#F8F7FB` . Full palette in `src/theme` .
Typography	Poppins (headings), DM Sans (body), Lilita One (DOMTAK wordmark).
Components	Use the shared components in `src/components` ; do not restyle per screen. Touch targets are at least 44 px; every interactive element has an accessibility label.
Status language	Labels and tones for booking, payment, quote, event, application, ticket and case states are centralised in `src/lib` (for example `STATUS_LABEL` , `PAYMENT_LABEL`). The API returns state codes; the app maps them to text.
Design reference	Final designs of all 136 screens on the design canvas (page “V1 · Wireframe & final”).

4.5 Platform specifics

Topic	Android	iOS	Web
Identifier	Package `de.domtak.app`	Bundle ID `de.domtak.app` ; iPhone only (`supportsTablet: false`)	Domain to be chosen (for example `app.domtak.de`)
Distribution	Google Play (AAB); internal APK via EAS preview	App Store; TestFlight for testing	Vercel (static SPA, all paths rewrite to `index.html`)
Camera	CAMERA permission (declared)	`NSCameraUsageDescription` (declared)	Browser camera permission (HTTPS required)
Location	To add: expo-location, “when in use” permission with a usage text (Android ACCESS_COARSE/FINE_LOCATION; iOS `NSLocationWhenInUseUsageDescription`)		Browser geolocation
Push	Firebase Cloud Messaging (FCM) via Expo push service	APNs key via Expo push service	Optional Web Push; in-app notifications otherwise
Maps	Google Maps SDK (API key)	Apple Maps or Google Maps	Google Maps JS API or MapLibre in a `map.web.tsx` variant
Payments	Stripe PaymentSheet (cards, Google Pay)	Stripe PaymentSheet (cards, Apple Pay)	Stripe Payment Element
Pro subscription	Google Play Billing	Apple In-App Purchase	Stripe Billing (section 8.6)
Token storage	expo-secure-store (Keystore)	expo-secure-store (Keychain)	httpOnly, Secure, SameSite cookie
Deep links	App Links for `https://<domain>` ; scheme `domtak://`	Universal Links (associated domains); scheme `domtak://`	Normal URLs

4.6 From demo to production: integration plan

1. **API client.** Add a typed API client (generated from the backend’s OpenAPI document) and a server-state library such as TanStack Query for caching, retries and pagination. Base URL from `EXPO_PUBLIC_API_URL` .
2. **Reads.** Replace imports from `src/data/mock.ts` with queries, one domain at a time (profiles, venues, events, posts, bookings…). Keep the `src/data/types.ts` shapes as the contract; the backend returns the same field names (camelCase JSON).
3. **Writes.** Replace each zustand action with a mutation that calls the matching endpoint (section 7). The server returns the new state; the client never computes a payment or booking state on its own.
4. **Auth.** Store the refresh token in expo-secure-store on mobile and in an httpOnly cookie on the web; keep the short-lived access token in memory. Refresh on 401. Respect “remember me”.
5. **Images.** Replace `ImgKey` image keys with URLs from the media service (CDN). Keep bundled assets only for brand art (logo, onboarding illustrations).
6. **Remove demo pieces** (appendix B): demo controls (DemoBar) on 10 screens, the admin demo switch, sample scanner codes, the fixed `TODAY` date and sample application shortcuts.
7. **Real-time and push.** Subscribe to the message and notification channels (section 5.6); register the push token after sign-in.
8. **States.** Every screen gets loading, empty, error and offline states for real network conditions. Mutations that move money use idempotency keys.
9. **Persistence.** Persist only the session, preferences and drafts locally. Clear user data on sign-out.

4.7 Native modules to add

Install with `npx expo install <package>` so versions match SDK 57, and check the versioned Expo docs (`docs.expo.dev/versions/v57.0.0`) before using an API. After adding a module with native code, Expo Go is no longer enough: use development builds (`eas build --profile development`).

Package	Purpose
`expo-notifications`	Push token registration, notification handling and deep links from notifications
`expo-location`	“Near me” on the map, Nearby feed and distance filters
`react-native-maps` (+ web map library)	Real map with event and venue pins
`expo-image-picker` (and `expo-image-manipulator`)	Photos for posts, stories, comments, avatars, venue listing, application documents
`@stripe/stripe-react-native`	PaymentSheet, Apple Pay, Google Pay (Expo config plugin)
In-app purchase library (for example `react-native-iap` or RevenueCat)	Pro subscription in the stores
`expo-secure-store`	Refresh token and device secrets
`expo-updates`	Over-the-air updates through EAS Update
`@sentry/react-native`	Crash and error reporting
`expo-localization` + i18n library	Prepared for German (the settings already offer English / Deutsch; V1 ships English)

4.8 Working on the app

```
cd mobile
npm ci                    # install exact versions from package-lock.json
npx expo start            # dev server: press a (Android), i (iOS simulator), w (web)
npm run typecheck         # tsc --noEmit
npm run lint              # ESLint (strict rules: no setState in effect bodies, no Date.now in
render)
npm run icons             # regenerate web icon paths after using a new icon name
npm run build:web         # static web build into mobile/dist
npx expo-doctor           # dependency and config check
```

Run typecheck and lint before every pull request. Never create or edit `android/` or `ios/` by hand; change `app.json` or a config plugin instead.

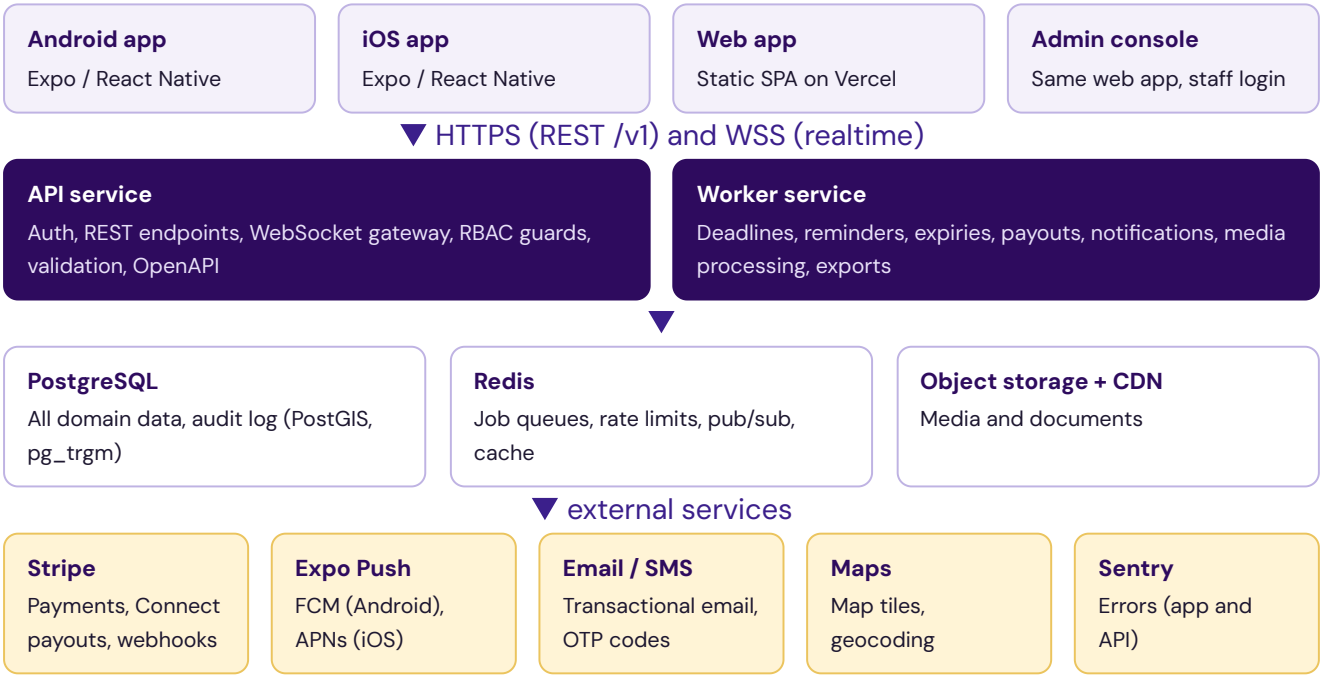
5. Backend architecture

No backend exists yet. This section is the recommended design. It is a modular monolith: one deployable API service plus a worker process, sharing one PostgreSQL database. This is the simplest architecture that meets V1's needs, and modules can be split out later if load requires it.

5.1 Recommended stack

Concern	Recommendation	Why
Runtime and language	Node.js 22 LTS, TypeScript	Same language as the app; the domain types in `mobile/src/data/types.ts` can be shared.
Framework	NestJS (REST controllers, WebSocket gateway, guards for RBAC, OpenAPI generation)	Clear module boundaries and built-in OpenAPI output for the typed app client.
Database	PostgreSQL 16 with PostGIS and pg_trgm	Relational integrity for money and bookings; geo queries for the map; fuzzy search.
ORM and migrations	Prisma or Drizzle; migrations committed to the repository	Reviewed, repeatable schema changes.
Jobs and cache	Redis + BullMQ (delayed jobs, retries)	24 h deadlines, 12 h reminders, expiries, payouts, notifications.
Files	S3-compatible object storage + CDN	Photos, avatars, application documents, data exports.
Hosting region	EU, Frankfurt (for example AWS eu-central-1)	GDPR: personal data of Berlin users stays in the EU.

5.2 System overview



5.3 Modules

Module	Responsibility
auth	Registration, email verification, sign-in, sessions and devices, password reset, two-step verification, change email/phone
users, profiles	Account, public profile (member, artist, band, DJ, venue, organizer), preferences, wishlist, follows, blocks
access	Workspaces, approved roles, professional applications and their review, staff permissions
social	Posts, stories, comments with photos, likes, saves, feeds (For You, Following, Nearby), communities, friends
messaging	Conversations, messages (text, image, voice), read state, booking-linked chats
events	Public and private events, lifecycle, line-up, ticket types, invitations, cost sharing, readiness and budget
tickets	Ticket orders, issued tickets, QR codes, check-in, refunds
providers	Artist services and pricing, lessons, availability, venue listings (parking, garden, catering, security, charges)
bookings	Booking requests, provider requests, quotes, change requests, cancellations, disputes, deadlines
payments	Payment intents, authorization and capture, refunds, failures and retries, webhooks, ledger, payouts, subscriptions
reviews	Event and venue star reviews; musician “would book again” reviews with praise tags (verified bookings only)
discovery	Search, filters, geo queries, sorting, promotions (sponsored placement)
notifications	In-app notifications, action queue, push, email, preferences
media	Upload URLs, image processing, CDN URLs, deletion
support	Support cases with reference numbers, linked records, messages, appeals
moderation, admin	Reports, decisions, removed content, admin queues, metrics, audit log
privacy	Data export, account deletion, retention jobs, consent records

5.4 Authentication and authorization

Authentication

- Email and password registration. Passwords hashed with Argon2id. Email verified with a 6-digit code (valid 10 minutes, at most 5 attempts, resend limited). The app’s `/verify` screen already expects a 6-digit code.
- Short-lived access token (JWT, about 15 minutes) and a rotating refresh token stored server-side per device. “Remember me” off = session ends when the app restarts. The devices list in Account & security reads the sessions table; signing out a device revokes its refresh token.
- Password reset, change email and change phone all use one-time codes. Two-step verification (`prefs.twoFactor`) asks for an SMS code (or TOTP) at each new sign-in.
- Rate limits on sign-in, code requests and code checks per account, IP and device.
- DECISION Additional sign-in methods (Google, Apple) and the age policy (blueprint F01–F04). If any third-party sign-in is offered on iOS, Sign in with Apple must be offered too (App Store rule).

Authorization

- The server stores `approved_roles` and the `active_role` per account. Switching (`PUT /v1/me/workspace`) is refused for a role that is not approved.
- Each endpoint declares the roles it accepts (guards) and checks ownership in the query (for example `WHERE provider_id = :me`). Never trust ids or roles sent by the client.
- Staff permissions (`admin` , `reviewer` , `support` , `finance` , `moderator`) live in a separate table, require two-step verification, and every staff decision writes an audit log entry (actor, action, target, note, time).

5.5 Background jobs and timers

Job	When	What it does
Provider reminder	12 h after a valid request reaches the provider	Push and email to the provider, only if the request is still awaiting a response.
Request expiry	24 h after the request reached the provider	Status `expired` ; payment `release_pending` and start the release with the payment provider; notify both sides.
Quote expiry	At the end of `validUntil`	Quote `expired` ; customer can ask for a new quote.
Authorization watch	Daily	Card authorizations expire after a limited time at the payment provider; capture or release before expiry and alert finance.
Payment reconciliation	On webhook and hourly	Move `capture_pending` , `release_pending` , `refund_pending` , `payout_pending` to their final state only when the provider confirms.
Event lifecycle	At event start and end	`in_progress` at start, `completed` at end; unlock reviews; start payout holds.
Reminders	24 h before an event; before a cost-sharing deadline	Ticket holders and guests who have not paid.
Payouts	After completion + hold period DECISION	Create transfers to providers (net of commission); mark `payout_pending` → `paid_out` on confirmation.
Promotions	At promotion end	Remove sponsored placement.
Application SLA	Hourly	Flag applications older than the 24 h review target in the admin queue.
Privacy	On request; nightly	Build data exports; execute account deletions; purge data past its retention period.

All jobs are idempotent (safe to run twice) and re-check the current state before acting, because a provider may answer one second before the expiry job runs.

5.6 Realtime, notifications and messaging

- One authenticated WebSocket connection per signed-in app instance. Server events: `message.created` , `conversation.read` , `notification.created` , `booking.updated` , `request.created` , `ticket.checked_in` . Use Redis pub/sub so several API instances can deliver events.
- When the user is offline, the same events go out as push notifications (Expo push service → FCM / APNs). Tapping a push opens its deep link (for example `/booking/b3`).
- The home attention banner and dashboard use an **action queue** endpoint that returns open items: payment failed, quote received, accepted and awaiting payment, change requested, event cancellation pending,

invitations, unpaid contributions, friend requests, support replies, application updates and (for providers) new requests.

Notice	Recipient	Channels	Type
New booking request, 12 h reminder	Provider	Push, email, in-app	Essential
Request accepted, declined or expired; quote received; change requested	Customer	Push, email, in-app	Essential
Payment failed, refund started or completed, payout sent	Payer / provider	Push, email, in-app	Essential
Event cancelled or changed	Ticket holders, guests, booked providers	Push, email, in-app	Essential
Application decision, support reply, moderation decision	Account	Push, email, in-app	Essential
New message	Recipient	Push, in-app	Optional (setting “Messages”)
Events near me	Members who opted in	Push	Optional (setting “Events near me”)
Likes, comments, follows, friend requests, invitations	Account	In-app, push	Social

5.7 Media

- The app asks for an upload URL (`POST /v1/media/uploads`), uploads directly to object storage, then sends the media id with the post, comment, avatar or listing.
- A worker checks the file type, strips location metadata (EXIF), creates sizes (thumbnail, feed, full) and publishes CDN URLs.
- Limits per kind are configuration, not code (file size, count per post or comment). Deleting a post or account deletes its media.
- DECISION Video posts and live streaming need a video service (for example Mux or Cloudflare Stream). V1 can launch with photos only and keep the video and live screens behind a feature flag.

5.8 Search and location

- Venues and events store a geographic point (PostGIS `geography(Point)`) set by geocoding the address. Distance filters use `ST_DWithin` ; results return `distanceKm` from the user’s location.
- Text search across events, artists, lessons, venues, communities, organizers and people uses PostgreSQL full-text search plus trigram similarity. A dedicated search engine is only needed later.
- DECISION Exact venue address visibility before booking (blueprint F06–F07). Until decided, return the area and approximate pin publicly and the exact address only to confirmed participants.

6. Data model

The front-end's domain types (`mobile/src/data/types.ts` and the interfaces in `src/store/useApp.ts`) are the contract. The tables below are the recommended PostgreSQL schema behind them. Table names are snake_case; the API returns camelCase JSON matching the TypeScript types.

Conventions. Primary keys are UUIDs. Every table has `created_at` and `updated_at` . Money is stored as integer cents with a currency (`EUR`), never as floating point; the app formats cents to euro. Timestamps are `timestamptz` in UTC; the app shows Europe/Berlin time. Soft-delete only where retention requires it (bookings, payments, tickets); otherwise delete.

One booking, two views. The demo keeps the customer's bookings (`Booking`) and the provider's incoming requests (`IncomingRequest`) as separate sample lists. In the backend they are the **same booking row**, returned from the customer side by `/v1/bookings` and from the provider side by `/v1/provider/requests` . A provider's counter offer becomes a quote revision on that booking.

6.1 Accounts, access and social

Table	Key columns	Notes
users	id, email (unique, case-insensitive), email_verified_at, phone, phone_verified_at, password_hash, active_role, approved_roles role[], two_factor_enabled, language, deleted_at	`approved_roles` always contains `fan` . Roles: fan, musician, venue, organizer.
sessions	id, user_id, device_name, platform, refresh_token_hash, remember, last_seen_at, revoked_at	Devices list; sign out other devices.
verification_codes	id, user_id, purpose, target, code_hash, expires_at, attempts, used_at	Purposes: verify_email, reset_password, change_email, change_phone, two_factor.
preferences	user_id, notify_bookings, notify_messages, notify_nearby, private_account, remember_me, language	Mirrors `Prefs` .
profiles	id, user_id, handle (unique), name, kind, title, bio, city, avatar_media_id, cover_media_id, genres text[], instruments text[], links jsonb, verified	Kinds: fan, musician, dj, band, organizer. Counters are derived or cached.
applications	id, user_id, role, status, details jsonb, missing text[], reason, submitted_at, decided_at, reviewer_id	Status: pending, info, approved, rejected. One open application at a time.
application_history	id, application_id, at, status, note, actor_id	Status timeline shown to the applicant.
staff_permissions	user_id, permission	admin, reviewer, support, finance, moderator.
follows, blocks	follower_id / followee_id; blocker_id / blocked_id	Unique pairs. Blocking removes friendship and pending requests.
friend_requests, friendships	id, from_id, to_id, status, responded_at; user_a, user_b	Status: pending, accepted, declined.

Table	Key columns	Notes
posts, post_media	id, author_id, kind, caption, location, event_id, track, visibility, removed_at; post_id, media_id, position	Only approved professionals create posts. Visibility: Public, Followers.
stories	id, author_id, frames (media ids), is_live, expires_at	Professionals only.
comments, comment_media, comment_likes	id, target_type (post or event), target_id, author_id, text, removed_at	Any member may attach photos to a comment.
post_likes, post_saves	user_id, post_id	
communities, community_members	id, name, description, cover_media_id, genres, rules; community_id, user_id	Created by the platform only in V1.
conversations, conversation_members, messages	id, booking_id; conversation_id, user_id, last_read_at; id, conversation_id, sender_id, type, text, media_id, duration_s, liked	Types: text, image, voice. Booking chats link to a booking.
notifications, push_tokens	id, user_id, type, actor_id, text, href, thumb_media_id, read_at; user_id, token, platform	
wishlist_items	user_id, kind (event, artist, venue), target_id	
media	id, owner_id, kind, storage_key, mime, width, height, bytes, variants jsonb, status	Status: uploading, ready, rejected.

6.2 Providers, events and tickets

Table	Key columns	Notes
venues	id, owner_id, name, type, area, city, postcode, address, location geography, capacity, rooms, stages, equipment, occasions text[], description, facilities text[], requirements text[], pricing_mode, price_per_night_cents, status	Types: Club, Open air, Hall, Loft, Garden, Bar.
venues (amenities)	parking jsonb {available, spaces, kind, notes}; garden jsonb {available, capacity, covered, notes}; catering jsonb {available, price_per_guest, min_guests, notes}; security jsonb {available, guests_per_guard, price_per_guard, required_from, notes}; extra_charges jsonb [{label, amount}]	Parking and garden are separate. Parking kinds: on-site, street, garage, nearby.
venue_media	venue_id, media_id, position	Cover first.
artist_services	profile_id, pricing_mode, fee_per_hour_cents, min_hours, radius_km, instant_book, travel_cents, travel_note, equipment_cents, equipment_note	Pricing: listed or quote.
availability	owner (profile or venue), weekdays jsonb, blocked_dates date[]	
lessons	id, profile_id, title, instrument, format, location, duration_min, price_cents, quote_only, level, description, active	Formats: in-person, online, at-studio.
events	id, title, subtitle, cover_media_id, category, genre, venue_id (nullable), address, location geography, host_id, organizer_id, is_private, state, start_at, end_at, age_limit, description, refund_policy, expected_guests, cancel_reason, cancelled_at	Only approved organizers may set `is_private = false` and publish.
event_lineup	event_id, profile_id or name, position	

Table	Key columns	Notes
ticket_types	id, event_id, name, description, price_cents, capacity, sales_start, sales_end	Sold count derived from tickets. Sold types cannot be deleted or repriced.
cost_sharing	event_id, enabled, per_guest_cents, deadline, note	Private events.
invitations	id, event_id, guest_id, status, contribution_cents, payment_state, ticket_id, sent_at, message	Status: invited, accepted, declined.
ticket_orders	id, buyer_id, event_id, total_cents, payment_id, status	
tickets	id, order_id, invite_id, event_id, type_id, holder_name, holder_id, code (unique), status, paid_cents, checked_in_at, checked_in_by	Status: valid, used, cancelled, refunded.
check_ins	id, event_id, ticket_id, code, result, scanned_at, scanned_by, device_id	Results: accepted, already_used, wrong_event, cancelled, refunded, unknown. Every scan is logged.
promotions	id, event_id, organizer_id, budget_cents, days, radius_km, audience text[], starts_at, ends_at, payment_id	Sponsored placement in feeds and lists.
reviews	id, author_id, target_type (event, venue, musician), target_id, booking_id, rating (1–5, null for musicians), recommend, tags text[], text, verified	Musician reviews need a completed booking (`verified = true`).

6.3 Bookings, payments and support

Table	Key columns	Notes
bookings	id, customer_id, type (musician, venue, lesson), provider_profile_id, venue_id, lesson_id, event_id, title, date, start_time, end_time, hours, location, guests, equipment, extras, note, pricing_mode, amount_cents, charges jsonb, status, payment_state, deadline_at, reminder_at, reminded_at, cancel_reason, pending_cancellation jsonb, payment_attempts	`charges` is a snapshot of the agreed price lines (base, travel, equipment, catering, security, fee) and never changes when the provider edits prices.
booking_timeline	id, booking_id, at, label, actor_id	Shown on the status screen and in admin.
booking_changes	id, booking_id, note, requested_by, requested_at, previous_status, resolved_at, accepted	Agreed terms stay until the other side accepts.
booking_evidence	id, booking_id, added_by, media_id or text	Disputes.
quotes	id, booking_id, version, status, items jsonb [{label, amount}], total_cents, valid_until, note	One row per version. Version 0 = requested (nothing priced).
payments	id, purpose (booking, tickets, contribution, promotion), purpose_id, amount_cents, currency, state, provider, provider_ref, capture_method, attempt_no, idempotency_key, failure_code	A retry is a new attempt on the same purpose; never a new booking.
refunds	id, payment_id, amount_cents, reason, state, provider_ref, decided_by	State: pending, succeeded, failed.
ledger_entries	id, booking_id, kind, amount_cents, at	Kinds: booking_value, service_fee, commission, refund, payout. Feeds earnings and the finance dashboard.

Table	Key columns	Notes
payout_accounts	user_id, provider_account_id, status	Connected account onboarding and verification status.
payouts	id, user_id, amount_cents, state, provider_ref, requested_at, paid_at	State: processing, paid, failed.
subscriptions	user_id, plan (monthly, yearly), store (apple, google, stripe), status, trial_end, period_end, store_ref	Pro plan.
support_cases, support_messages	id, ref (for example DT-4821), user_id, topic, linked_kind, linked_id, status, resolved_at; id, case_id, from (user, support), author_id, text, at	Status: open, waiting (support replied), resolved.
reports	id, reporter_id, kind, target_id, reason, details, status, decided_by, decision_note, decided_at	Kinds: post, comment, profile, message, event. Status: open, dismissed, restricted, removed. One open report per reporter and target.
audit_log	id, at, actor_id, action, target_type, target_id, note, before jsonb, after jsonb	Append-only. Every staff decision and money movement.

6.4 Booking lifecycle

Listed price (happy path):

requested → pending → accepted → confirmed → completed

Exceptions: declined, expired, cancelled, disputed, change_requested, payment_failed

From	Trigger	To	Payment state and side effects
—	Customer submits a listed-price request	requested	Payment step starts.
requested	Valid request reaches the provider	pending	`authorized` (if the payment model uses authorization); `deadline_at = +24 h`, `reminder_at = +12 h`.
pending	Provider accepts	accepted	`capture_pending`; capture starts. Not confirmed yet.
accepted	Payment verified (webhook)	confirmed	`paid`; chat and calendar entries for both sides.
accepted	Payment fails	payment_failed	`failed`; customer retries on the same booking (back to accepted / capture).
pending	Provider declines (reason)	declined	`release_pending` → `released` when confirmed.
pending	24 h deadline passes	expired	`release_pending` → `released`. Never shown as released before confirmation.
any open	Customer or provider cancels (reason)	cancelled	Authorization → `release_pending`; captured payment → `refund_pending` → `refunded`, per the cancellation policy.
confirmed	Either side requests a change	change_requested	Payment unchanged; on accept, new terms apply; on decline, back to the previous status.

From	Trigger	To	Payment state and side effects
confirmed / completed	Problem reported	disputed	Payment state stays visible; a support case opens.
confirmed	Event date has passed	completed	Payout scheduled: `payout_pending` → `paid_out`. Reviews unlock.
linked event	Host cancels the event	(unchanged)	`pending_cancellation` set; the booking goes through the cancellation process, it is not cancelled automatically.

The legacy status `paid` (payment verified, not yet confirmed) is kept for compatibility; new bookings move straight from accepted to confirmed when payment is verified. The client mapping is `paymentAfter()` in `mobile/src/lib/booking.ts`.

Quote-based bookings

quote requested → **quote sent (v1)** → **accepted · payment** → **confirmed** → **completed**

- Requesting a quote charges nothing. The provider sends an itemised quote with a validity date; each revision increases the version and needs the customer's acceptance again.
- Accepting a valid quote moves the booking to `accepted` and starts the payment. An expired or unsent quote cannot be accepted. Declining ends the offer without a booking.
- DECISION Quote validity length, the provider deadline for quotes, and whether customer acceptance replaces provider confirmation (blueprint F15–F16).

6.5 Payment states

State	Meaning
not_started	No payment step yet (for example a quote request).
authorized	Amount reserved, pending the provider's decision.
capture_pending	Provider accepted; capture requested, not yet confirmed.
paid	Payment verified by the payment provider.
release_pending / released	Authorization release requested / confirmed (declined, expired or cancelled before capture).
refund_pending / refunded	Refund requested / confirmed by the payment provider.
failed	The payment did not succeed. Nothing was charged.
payout_pending / paid_out	Provider payout requested / confirmed.
disputed	A case or chargeback is open.

Only payment-provider webhooks (or a reconciliation job reading the provider's API) move a `*_pending` state to its final state.

6.6 Other lifecycles

Entity	States and rules
Event	`draft` → `planning` (provider bookings outstanding) → `published` → `ready` (all linked bookings confirmed) → `in_progress` (from start time) → `completed` (after end time); `cancelled` at any time with a reason. Private published events read "Invitations open". Readiness and budget come from the linked bookings (`lib/events.ts`).
Application	`pending` → `approved` (role added to approved_roles) | `info` (missing items listed; applicant resubmits → pending) | `rejected` (reason; applicant may apply again).
Ticket	`valid` → `used` (first check-in time stored) | `cancelled` | `refunded` . A ticket of a cancelled event is refused at the door.
Invitation	`invited` → `accepted` | `declined` . Free invitation: ticket issued on accept. With cost sharing: ticket issued only after the contribution payment succeeds.
Support case	`open` (waiting for support) ⇄ `waiting` (support replied, waiting for the member) → `resolved` . Cannot resolve while a linked refund or release is pending.
Report	`open` → `dismissed` | `restricted` | `removed` (posts and comments are hidden for everyone). Decision note and audit entry required.
Friend request	`pending` → `accepted` | `declined` ; the sender may cancel while pending.

6.7 Pricing and scoring rules

Rule	Formula (from `mobile/src/lib`)
Catering	`pricePerGuest × max(guests, minGuests)` , only if available and guests > 0
Security	`ceil(guests / guestsPerGuard) × pricePerGuard` ; mandatory when guests ≥ `requiredFrom`
Artist estimate	Base = fee per hour × hours (no base price when the artist prices by quote); travel and equipment are separate lines; optional service fee line. The provider's minimum hours (availability) are enforced on the request.
Lesson estimate	Price per session × sessions (its own price, never the performance fee); quote-only lessons have no price
Customer service fee	5% of the booking total (demo rate) DECISION
Platform commission	10% of each booking paid through the app, deducted from the provider's payout (as stated in the Terms) DECISION final rate and who pays
Provider net	`gross × (1 − commission rate)`
Musician score	`round(recommended / total × 100)` % over verified reviews; top praise tags by count; no score when total = 0
Venue / event rating	Average of 1–5 star reviews with count; show "New" when there are no reviews
Deadlines	Response window 24 h, reminder 12 h (`RESPONSE_HOURS` , `REMINDER_HOURS`)

Rates and windows must be server configuration (not constants in code), because several are open decisions.

7. API specification (REST v1)

7.1 Conventions

Topic	Rule
Base URL	`https://api.<domain>/v1`. JSON in and out, camelCase field names matching `mobile/src/data/types.ts`.
Authentication	Mobile: `Authorization: Bearer <access token>`. Web: httpOnly session cookie plus a CSRF token header on writes.
Money	Integer euro cents in every money field (`amount`, `price`, `total` …), with `currency: "EUR"` on payment objects.
Time	ISO 8601 in UTC (`2026-10-24T18:00:00Z`); dates as `YYYY-MM-DD`.
Lists	Cursor pagination: `?limit=20&cursor=…` → `{ items: [...], nextCursor }`.
Errors	RFC 7807 problem JSON: `{ type, title, status, detail, code, fields }`. Stable codes the app can map to text, for example `ROLE_NOT_APPROVED`, `BOOKING_STATE_CONFLICT`, `QUOTE_EXPIRED`, `DEADLINE_PASSED`, `TICKET_ALREADY_USED`, `WRONG_EVENT`, `PAYMENT_DECLINED`.
State changes	Actions are explicit endpoints (`POST /bookings/:id/cancel`), not free PATCHes of `status`. Each returns the updated resource. Concurrent changes return `409`.
Idempotency	`Idempotency-Key` header is required on every request that creates a booking, order, payment, refund or payout. Repeating the key returns the first result.
Rate limits	`429` with `Retry-After`. Strict limits on auth, codes, search and reports.
Contract	OpenAPI 3.1 document at `/v1/openapi.json`; the app's API client is generated from it in CI.

The third column names the store action in `mobile/src/store/useApp.ts` (or other store file) that the endpoint replaces.

7.2 Auth and account

Endpoint	Purpose	Replaces
`POST /auth/register`	Create a member account; sends an email code	`register`
`POST /auth/verify-email`, `POST /auth/resend-code`	Verify the 6-digit code	verify screen
`POST /auth/login`, `POST /auth/2fa/verify`	Sign in (with remember me); second step if enabled	`signIn`
`POST /auth/refresh`, `POST /auth/logout`	Rotate tokens; end session	`signOut`
`POST /auth/password/forgot` · `/verify` · `/reset`	Password reset in three steps	forgot-password screen
`GET /me`, `PATCH /me`	Current account and profile	`updateUser`
`PUT /me/workspace`	Switch active workspace (approved roles only)	`switchWorkspace`

Endpoint	Purpose	Replaces
`GET /me/preferences`, `PATCH /me/preferences`	Notification, privacy, language preferences	`setPref`
`POST /me/email` + `/confirm`; `POST /me/phone` + `/confirm`	Change email or phone with a code	account-security
`POST /me/password`; `POST /me/2fa`, `DELETE /me/2fa`	Change password; two-step verification	account-security
`GET /me/sessions`, `DELETE /me/sessions/:id`	Devices; sign out a device	account-security
`POST /me/export`, `DELETE /me`	GDPR data export; delete account	account-security, settings
`POST /me/push-tokens`, `DELETE /me/push-tokens/:token`	Register a device for push	new

7.3 Professional access

Endpoint	Purpose	Replaces
`POST /applications`	Submit an application (role, details, document media ids)	`submitApplication`
`GET /applications/current`, `PUT /applications/current`	Status, missing items, history; resubmit	application screen

7.4 Social

Endpoint	Purpose	Replaces
`GET /feed?tab=forYou|following|nearby`, `GET /stories`	Home feed pages and stories row (blocked accounts excluded)	`useFeedPosts`
`POST /posts`, `DELETE /posts/:id`	Create (approved professionals only) or delete a post	`addPost`
`POST /stories`	Create a story (professionals only)	new-post
`GET /posts/:id`; `POST|DELETE /posts/:id/like`; `POST|DELETE /posts/:id/save`	Post detail, like, save	`toggleLike`, `toggleSave`
`GET|POST /posts/:id/comments`; `GET|POST /events/:id/comments`	Comments with optional photo media ids	`addComment`
`POST|DELETE /comments/:id/like`, `DELETE /comments/:id`	Comment like; delete own comment	`toggleCommentLike`
`GET /profiles/:id`, `/profiles/:id/posts`, `/followers`, `/following`	Profile and connections	mock reads
`POST|DELETE /profiles/:id/follow`	Follow / unfollow	`toggleFollow`
`POST|DELETE /profiles/:id/block`, `GET /me/blocked`	Block (ends friendship) / unblock	`toggleBlock`
`GET /friends`, `DELETE /friends/:profileId`	Friends list; remove	`removeFriend`

Endpoint	Purpose	Replaces
`GET|POST /friend-requests` ; `POST /friend-requests/:id/accept|decline` ; `DELETE /friend-requests/:id`	Send, answer, cancel	`sendFriendRequest` , `respondFriendRequest` , `cancelFriendRequest`
`GET /communities` , `GET /communities/:id` , `POST|DELETE /communities/:id/membership`	Communities; join / leave	`toggleCommunity`
`GET /wishlist` , `PUT|DELETE /wishlist/:kind/:id`	Saved events, artists, venues	`toggleWish`

7.5 Messages and notifications

Endpoint	Purpose	Replaces
`GET /conversations` , `POST /conversations`	List; open (or reuse) a conversation with someone	`ensureConversation`
`GET|POST /conversations/:id/messages`	Messages (text, image, voice)	`sendMessage`
`POST /conversations/:id/read` , `POST /messages/:id/like`	Read state; like a message	`markConversationRead` , `toggleMessageLike`
`GET /notifications` , `POST /notifications/read`	Notifications; mark read	`markNotificationsRead`
`GET /action-queue`	Open items for the attention banner and dashboard	`useActionQueue`
`WSS /v1/realtime`	Realtime events (section 5.6)	new

7.6 Discovery, providers and reviews

Endpoint	Purpose	Replaces
`GET /search?q=&type=`	Events, artists, lessons, venues, communities, organizers, people	search screen
`GET /events?category&date&free&priceMax&genres&area&lat&lng&radiusKm&sort`	Discoverable events with filters and distance	`applyFilters`
`GET /events/map?lat&lng&radiusKm&…`	Map pins for nearby events (and venues on request)	map screen
`GET /events/:id` ; `POST|DELETE /events/:id/going`	Event detail; going	`toggleGoing`
`GET /artists?kind&genre&sort` , `GET /artists/:id` , `GET /artists/:id/reviews`	Artists with services, lessons, charges and “would book again” score	`useArtistScore`
`GET /venues?type&parking&garden&catering&security` , `GET /venues/:id` , `GET /venues/:id/reviews`	Venues with separate parking and garden, catering, security, charges	`useVenue`
`POST /venues/:id/estimate` ; `POST /artists/:id/estimate`	Server-side price lines for the request form (guests, catering, security, extras, travel, equipment)	`venueEstimate` , `artistEstimate`

Endpoint	Purpose	Replaces
`GET /lessons?instrument&format&level`, `GET /lessons/:id`	Lessons	`useLesson`
`POST /bookings/:id/review`, `POST /events/:id/review`	Review after completion: stars, or recommend + praise tags for musicians	`submitReview`

7.7 Events, invitations and tickets

Endpoint	Purpose	Replaces
`POST /events`, `PATCH /events/:id`	Create (draft) and edit; sold ticket types are protected	`addEvent`, `saveEventDraft`, `updateEvent`
`POST /events/:id/publish`	Publish (public: approved organizers only)	`setEventState`
`POST /events/:id/cancel`	Cancel with reason; notifies everyone; flags linked bookings	`cancelEvent`
`GET /me/events`, `GET /events/:id/workspace`	Hosted events; readiness, budget, providers, guests	`readiness`, `budget`
`POST /events/:id/bookings/:bookingId`	Attach a provider booking to an event	`linkBooking`
`POST /events/:id/invitations`, `PUT /events/:id/cost-sharing`	Invite guests; configure cost sharing	`inviteGuests`, `setCostSharing`
`GET /invitations/:id`, `POST /invitations/:id/respond`, `POST /invitations/:id/pay`	Guest RSVP and contribution (ticket only after successful payment)	`respondInvite`, `payContribution`
`POST /ticket-orders`	Reserve tickets and create the payment	`startTicketCheckout`, `completeCheckout`
`GET /me/tickets`, `GET /tickets/:id`	Wallet and e-ticket (QR payload)	tickets
`POST /tickets/:id/refund`	Refund under the event's policy (organizer or support)	`refundTicket`
`POST /events/:id/check-ins`	Validate a scanned code; returns result and previous check-in time	`checkIn`
`GET /events/:id/attendees`	Guest list with check-in state	attendees screen
`POST /events/:id/promotions`	Paid promotion (budget, days, radius, audience)	`promoteEvent`

7.8 Bookings (customer side)

Endpoint	Purpose	Replaces
`POST /bookings`	Listed-price request (artist, venue or lesson); server computes price lines and starts the payment step	`createBooking`
`POST /bookings/quote-requests`	Request a quote (nothing charged)	`requestQuote`
`GET /bookings?tab=musicians|venues|lessons`, `GET /bookings/:id`	List and detail with timeline, deadline, payment state	my-bookings, booking screen

Endpoint	Purpose	Replaces
`POST /bookings/:id/quote/accept`, `/quote/reject`	Answer the current quote version	`acceptQuote`, `rejectQuote`
`POST /bookings/:id/payment`	Start or retry payment on the same booking	`startBookingCheckout`, `retryPayment`
`POST /bookings/:id/changes`, `POST /bookings/:id/changes/:changeId/resolve`	Request a change; accept or decline it	`requestChange`, `resolveChange`
`POST /bookings/:id/cancel`	Cancel with reason; returns the money outcome	`cancelBooking`
`POST /bookings/:id/dispute`, `POST /bookings/:id/evidence`	Report a problem; add evidence	`setBookingStatus('disputed')`, `addBookingEvidence`

7.9 Provider workspace

Endpoint	Purpose	Replaces
`GET /provider/requests?status=`, `GET /provider/requests/:id`	Incoming requests with deadline, reminder state, date conflicts	`requestConflict`
`POST /provider/requests/:id/accept`, `/decline`	Answer within the deadline (decline needs a reason)	`setRequestStatus`
`POST /provider/requests/:id/quote`	Send or revise an itemised quote (also used for counter offers)	`quoteRequest`, `counterRequest`
`GET|PUT /provider/availability`	Weekdays, blocked dates, fee, minimum hours, radius, instant book	`setAvailability`
`GET|PUT /provider/services`; `GET|POST|PATCH|DELETE /provider/lessons`	Pricing mode, travel and equipment charges; lessons	`store/services.ts`
`GET|PUT /provider/venue-listing`, `POST /provider/venue-listing/publish`	Venue listing editor and publish	`updateVenueListing`
`GET /provider/earnings`, `GET|POST /provider/payouts`	Ledger (gross, commission, net, refunds), payouts	`requestPayout`
`POST /provider/payout-account/link`	Onboarding link for the payout account (identity and bank verification)	new

7.10 Payments, subscriptions, media and support

Endpoint	Purpose	Replaces
`GET /payment-methods`, `POST /payment-methods/setup`, `DELETE /payment-methods/:id`, `PUT /payment-methods/default`	Saved methods through the payment provider (no card data on our servers)	`addPaymentMethod`, `setSelectedPayment`
`POST /webhooks/stripe`	Payment provider events (signature verified)	new
`GET /subscription`, `POST /subscription/store-receipt`, `POST /subscription/checkout`	Pro plan status; verify App Store / Play purchase; web checkout	`setPlan`

Endpoint	Purpose	Replaces
`POST /webhooks/app-store`, `POST /webhooks/google-play`	Store subscription notifications (renewals, cancellations, refunds)	new
`POST /media/uploads`, `POST /media/:id/complete`	Upload URL; confirm upload	new
`GET|POST /support/cases`, `GET /support/cases/:id`, `POST /support/cases/:id/messages`, `POST /support/cases/:id/close`	Support cases with linked record	`openCase`, `replyCase`, `closeCase`
`POST /reports`	Report content or an account (one open report per target)	`reportContent`

7.11 Admin (staff permissions)

Endpoint	Purpose	Permission
`GET /admin/metrics`	Overview metrics and queues	any staff
`GET /admin/applications`, `POST /admin/applications/:id/decision`	Approve, request information (missing items), reject (reason)	reviewer
`GET /admin/bookings?q=`, `GET /admin/bookings/:id`, `GET /admin/requests/:id`	Investigation: timeline, participants, payment state, exceptions	support, finance
`POST /admin/bookings/:id/expire`	Expire an overdue request	support
`GET /admin/finance/queue`, `POST /admin/payments/:id/reconcile`, `POST /admin/refunds`	Captures, releases, refunds, payouts; authorized refund decisions	finance
`GET /admin/reports`, `POST /admin/reports/:id/decision`	Dismiss, restrict, remove	moderator
`GET /admin/support/cases`, `POST /admin/support/cases/:id/reply`	Answer and resolve cases	support
`GET /admin/audit-log`	Decision history	admin

8. Payments

Open decision first. The blueprint leaves the payment provider, the legal payment structure, and the choice between "authorize now, capture on acceptance" and "charge after acceptance" open (F14–F19). The app is built so that either works: it shows the payment state of each booking and never promises escrow. The design below uses Stripe as the recommended provider and supports both models through configuration. Have the model confirmed by legal and finance before building it.

8.1 Recommended model

- **Provider:** Stripe, with **Stripe Connect** (Express accounts) for artists, venues and organizers. Stripe handles identity verification (KYC) and bank payouts for providers. Card data never touches Domtak servers.
- **Flow of money:** the customer pays Domtak's platform account; after the service is delivered, Domtak transfers the provider's share (booking value minus commission) to the provider's connected account ("separate charges and transfers"). This lets payouts wait until the event is completed.
- **Payment UI:** Stripe PaymentSheet on Android and iOS (cards, Google Pay, Apple Pay) and the Payment Element on the web. The app shows Visa, Google Pay, PayPal and SEPA direct debit as options today; enable each method in Stripe and check which methods support the chosen model (for example, SEPA direct debit does not support an authorization hold).
- **Source of truth:** Stripe webhooks. The API never marks money as moved until the matching webhook (or a reconciliation read) confirms it.

8.2 Listed-price booking

#	Step	Model A: authorize, capture on acceptance	Model B: charge after acceptance
1	Customer submits the request	Server creates the booking (`requested`) and a PaymentIntent with manual capture; app confirms it in the PaymentSheet	Server creates the booking; no payment yet (`not_started`)
2	Valid request reaches the provider	On the "amount capturable" webhook: `pending` / `authorized` ; 24 h deadline and 12 h reminder scheduled	`pending` ; deadline and reminder scheduled
3	Provider accepts	Server captures: `accepted` / `capture_pending` → on success webhook `confirmed` / `paid`	`accepted` ; customer pays (PaymentIntent, automatic capture) → `confirmed` / `paid`
4	Provider declines, deadline passes, or cancellation before payment	Server cancels the PaymentIntent: `release_pending` → `released` on webhook	Nothing to release
5	Payment fails	`payment_failed` / `failed` ; the customer retries on the same booking (new attempt, same booking id, new idempotency key)	

#	Step	Model A: authorize, capture on acceptance	Model B: charge after acceptance
6	Event completed	`completed` ; payout scheduled after the hold period: `payout_pending` → `paid_out` (transfer to the connected account)	

Card authorizations are only valid for a limited time (typically about 7 days for online card payments). The 24-hour response window fits inside it; the authorization watch job (section 5.5) protects edge cases such as late change requests.

8.3 Quotes, tickets and contributions

- **Quotes:** requesting and receiving a quote charges nothing. Accepting a valid quote starts a normal payment (automatic capture) for the quote total; success confirms the booking.
- **Tickets:** `POST /ticket-orders` holds the requested ticket stock for a short time, creates a PaymentIntent and issues tickets (unique codes) only after payment success. Releasing unpaid holds is a job. DECISION Public ticket selling scope, fees and organizer settlement (blueprint F12, F20–F22).
- **Cost-sharing contributions:** one PaymentIntent per guest invitation; the guest ticket is issued only on success. DECISION Contribution limits and how the host receives the money.
- **Promotions:** paid by the organizer before the promotion starts.

8.4 Cancellations, refunds and disputes

- Before capture: release the authorization (no charge). After capture: create a refund according to the policy shown on the booking or event; the state stays `refund_pending` until Stripe confirms.
- Support may only promise a refund after an authorized decision (finance permission); every refund decision is in the audit log.
- Card disputes (chargebacks) set the booking or order to `disputed` ; evidence is collected from the booking (timeline, messages, check-ins, evidence files) and submitted through Stripe.
- DECISION Cancellation, refund, no-show, rescheduling and dispute policies (F13, F18, F20–F22).

8.5 Fees, commission and payouts

- Demo rates in the app: customer service fee 5% (`SERVICE_FEE_RATE`), platform commission 10% deducted from the provider (`PLATFORM_COMMISSION_RATE` , stated in the Terms). Both are server configuration. DECISION Final rates, who pays the commission, tax presentation and payment fee allocation.
- The ledger records booking value, service fee, commission, refunds and payouts separately, so provider earnings and the admin finance dashboard can show them separately (blueprint requirement).
- Providers must finish payout onboarding (identity and bank account) before their first payout. DECISION Payout timing and hold period.

8.6 Pro subscription

The Pro plan costs €9,99 per month or €89 per year, with the first month free.

- Inside the Android and iOS apps, a subscription that unlocks digital features must be sold through **Google Play Billing** and **Apple In-App Purchase** (store rules). On the web, use **Stripe Billing**. A service such as RevenueCat can combine the three into one entitlement.

- The server stores one subscription record per account and updates it from App Store Server Notifications, Google Play real-time developer notifications and Stripe webhooks.
- Bookings, event tickets and contributions are payments for real-world services and use Stripe in all apps. Check the store rules for paid event promotions, which may count as a digital purchase.

9. External services to activate

Each service below needs an account owned by the company (not by an individual developer), with access granted to the team through roles. Sign a data processing agreement (DPA) with every service that receives personal data, and record it in the privacy documentation.

Service	Used for	Setup steps	Keys and configuration
Domain and DNS	Web app, API, email, deep links	Register or confirm the domain (for example `domtak.de`). Create `app.`, `api.`, `media.` subdomains. Email DNS: SPF, DKIM, DMARC. Host `/.well-known/apple-app-site-association` and `/.well-known/assetlinks.json` for Universal Links and App Links.	DNS records
GitHub	Code, CI/CD	Repository `DOMTAK-V1`; protect `main` (reviews, required checks); add an API repository or `api/` folder; Actions secrets per environment.	`EXPO_TOKEN`, cloud deploy credentials
Expo (EAS)	Builds, store submission, OTA updates, push service	Create an Expo organization; run `npx eas-cli@latest init` in `mobile/` (adds the project id); configure build credentials; add channels for EAS Update.	EAS project id; `EXPO_TOKEN` for CI
Apple Developer Program	iOS app, push (APNs), Apple Pay, IAP	Enroll as an organization (needs a D-U-N-S number). In App Store Connect create the app `de.domtak.app`. Create an APNs key (upload through EAS). Apple Pay merchant id for Stripe. Subscription products with a one-month free trial. Associated domains.	Team id, APNs key, App Store Connect API key (for `eas submit`)
Google Play Console	Android app, Play Billing	Register as an organization; create the app `de.domtak.app`; enable Play App Signing; create a service account for automated uploads; subscription products; data safety form; content rating; internal testing track.	Service account JSON (for `eas submit`)
Firebase	Android push (FCM)	Create a project; add the Android app; generate an FCM v1 service account key and upload it to EAS credentials so the Expo push service can reach Android devices.	FCM service account key
Stripe	Payments, Connect payouts, web subscriptions	Create the account (Germany); activate Connect (Express); set branding; enable payment methods; verify the Apple Pay web domain; add webhook endpoints for staging and production; configure Billing products for the web Pro plan; keep test and live mode separate.	Publishable key (app), secret key and webhook secret (API)
Cloud hosting (recommended AWS, Frankfurt)	API, worker, database, cache, files	Accounts for staging and production; network (VPC); container service for API and worker; PostgreSQL 16 with PostGIS; Redis; S3 buckets with a CDN; secrets manager; logging and alarms; automated backups.	Database URL, Redis URL, bucket names, CDN domain
Transactional email (for example Amazon SES or Postmark)	Codes, receipts, booking and event notices	Verify the sending domain (DKIM); request production sending; create templates (verify email, reset, booking updates, tickets, receipts, support replies).	API credentials, from-address

Service	Used for	Setup steps	Keys and configuration
SMS (for example Twilio Verify)	Phone verification, two-step codes	Create a Verify service; register the sender for Germany; set fraud limits.	Account SID, auth token, service id
Google Maps Platform	Maps (Android, web), geocoding	Create a project with billing; enable Maps SDK for Android, Maps SDK for iOS (if used), Maps JavaScript API and Geocoding API; create separate restricted keys (Android package + signing certificate fingerprint, iOS bundle id, web referrer, server IP).	Maps keys per platform, server geocoding key
Sentry	Crash and error reporting	Projects for app and API; upload source maps from EAS builds and EAS updates; alert rules.	DSNs, auth token for uploads
Vercel	Web app hosting	Project from `DOMTAK-V1` ; Root Directory empty or `mobile` ; framework preset “Other”; custom domain; environment variables per environment.	`EXPO_PUBLIC_*` variables
Product analytics (optional, for example PostHog EU)	Funnels and dashboard metrics	EU hosting; track only after consent; no personal data in event names.	Project key
Video (optional, for example Mux)	Video posts and live streaming	Only if video and live are in V1 scope (section 5.7).	Token id and secret
In-app purchases (optional, RevenueCat)	One entitlement across Apple, Google and Stripe	Connect App Store, Play and Stripe; configure the Pro entitlement.	Public SDK keys; webhook secret

10. Environments, deployment and release

10.1 Environments

Environment	Backend	Apps	Payments and data
Development	Local Docker Compose (API, worker, PostgreSQL, Redis, local S3) or a shared dev API	Development builds (`eas build --profile development`) and `npx expo start` ; web on localhost	Stripe test mode; seed data
Staging	Same setup as production, smaller	Preview builds (internal APK, TestFlight), EAS Update channel `preview` , Vercel preview URL	Stripe test mode; anonymised or seed data
Production	Frankfurt region, at least 2 API instances, separate worker	Store builds, EAS Update channel `production` , web on the main domain	Stripe live mode; real data

The sample data in `mobile/src/data/mock.ts` can be turned into a seed script for development and staging so testers see the same scenarios as the design (bookings b1–b7, requests r1–r6, invitation gi1, support case DT-4821…).

10.2 Configuration

App (build-time)

Variables prefixed `EXPO_PUBLIC_` are compiled into the app and are visible to anyone. Never put secrets in them. Convert `app.json` to `app.config.ts` so values such as the Maps keys and the API URL can differ per environment.

Variable	Purpose
`EXPO_PUBLIC_API_URL` , `EXPO_PUBLIC_WS_URL`	API and realtime endpoints
`EXPO_PUBLIC_STRIPE_PUBLISHABLE_KEY` , `EXPO_PUBLIC_APPLE_MERCHANT_ID`	Stripe client SDK
`EXPO_PUBLIC_SENTRY_DSN`	Error reporting
`EXPO_PUBLIC_WEB_URL`	Share links and deep links
Maps keys (in `app.config.ts` : Android and iOS native config; web variable)	Maps SDKs
`EXPO_PUBLIC_ENV`	`development` , `staging` or `production` (labels, feature flags)

Backend (runtime, from the secrets manager)

Variable	Purpose
`DATABASE_URL` , `REDIS_URL`	PostgreSQL and Redis
`JWT_SIGNING_KEY` , `REFRESH_TOKEN_PEPPER` , `CODE_PEPPER`	Tokens and one-time code hashing
`STRIPE_SECRET_KEY` , `STRIPE_WEBHOOK_SECRET`	Payments

Variable	Purpose
`APPLE_*` (App Store Server API key), `GOOGLE_PLAY_*` (service account)	Subscription verification
`S3_BUCKET`, `CDN_BASE_URL`	Media
`EMAIL_*`, `TWILIO_*`	Email and SMS
`EXPO_ACCESS_TOKEN`	Sending push through the Expo push service (enhanced security)
`GOOGLE_GEOCODING_KEY`	Address to coordinates
`SENTRY_DSN`	API errors
`COMMISSION_RATE`, `SERVICE_FEE_RATE`, `RESPONSE_HOURS`, `REMINDER_HOURS`, `PAYOUT_HOLD_DAYS`	Business settings (open decisions)

10.3 Backend deployment

1. CI builds one Docker image per commit; the same image runs the API and (with another command) the worker.
2. Merges to `main` deploy to staging automatically. Production deploys from a release tag after manual approval.
3. Database migrations run as a separate step before the new version starts. Use expand-then-contract changes so the old and new versions both work during a deploy.
4. Health endpoints: `/health/live` (process up) and `/health/ready` (database and Redis reachable). Roll back automatically if the new version is not ready.
5. Run at least two API instances behind a load balancer with TLS; scale on CPU and latency. WebSocket connections need sticky sessions or the Redis adapter.
6. Backups: daily automated database snapshots plus point-in-time recovery; test a restore before launch and every quarter. Object storage versioning on media.
7. Monitoring: error rates and latency per endpoint, job queue depth and failures, webhook failures, payment reconciliation mismatches, push and email delivery failures. Alerts go to an on-call channel.

10.4 Web app

- Already configured: `vercel.json` at the repository root and in `mobile/` installs with `npm ci`, builds with `npm run build:web` (Expo static export to `mobile/dist`) and rewrites every path to `index.html` (single-page app).
- Vercel project settings: Root Directory empty (repository root) or `mobile`; Framework Preset “Other”. Set the `EXPO_PUBLIC_*` variables for Preview and Production.
- Add security headers in `vercel.json` (Content-Security-Policy allowing the API, Stripe and Maps domains; Strict-Transport-Security; X-Content-Type-Options; Referrer-Policy) and a cookie consent banner before any non-essential tracking.
- The admin console is part of the web app and is only reachable for staff accounts (server-enforced).

10.5 Android release

```
# one-time: npx eas-cli@latest init ; npx eas-cli@latest credentials -p android
npx eas-cli@latest build -p android --profile preview      # internal APK for testers
npx eas-cli@latest build -p android --profile production   # AAB, versionCode auto-incremented
npx eas-cli@latest submit -p android --latest              # upload to the Play Console
```

- Release through Play Console tracks: internal testing → closed testing → production (staged rollout).
- Complete the store listing, data safety form (location, photos, payments, messages, identifiers), content rating, target audience, and the account deletion URL (the app already has in-app deletion).
- Keep Play App Signing enabled; EAS manages the upload keystore. Back up credentials in the company account.

10.6 iOS release

```
npx eas-cli@latest build -p ios --profile production        # EAS creates certificates and profiles
npx eas-cli@latest submit -p ios --latest                    # upload to App Store Connect / TestFlight
```

- Test through TestFlight (internal, then external testers) before App Review.
- App Review needs: a working reviewer account with sample data, privacy "nutrition label" answers, purpose strings for camera (present) and location (to add), in-app account deletion (present), the Pro plan as an in-app purchase, and Sign in with Apple if any other third-party sign-in is offered.
- `supportsTablet` is `false`: the app ships for iPhone only.

10.7 Build profiles and over-the-air updates

Recommended `eas.json` (extends the current file):

```
{
  "cli": { "version": ">= 16.0.0", "appVersionSource": "remote" },
  "build": {
    "development": { "developmentClient": true, "distribution": "internal", "channel": "development" },
    "preview":     { "distribution": "internal", "channel": "preview", "android": { "buildType": "apk" } },
    "production":  { "autoIncrement": true, "channel": "production", "android": { "buildType": "app-bundle" } }
  },
  "submit": { "production": {} }
}
```

- Add `expo-updates` and set a runtime version policy so an update only reaches builds with the same native code.
- JavaScript-only fixes: `npx eas-cli@latest update --channel production`. Changes to native modules or `app.json` need a new store build.
- Check the Expo SDK 57 documentation for the exact `expo-updates` and `eas.json` options before applying.

10.8 CI/CD pipelines (GitHub Actions)

Workflow	Trigger	Steps
app-ci	Pull request	`npm ci`, typecheck, lint, unit tests, `expo-doctor`, web build
app-preview	Merge to `main`	EAS Update to `preview`; Vercel preview deploy
app-release	Release tag	EAS production builds (Android, iOS) and submit; Vercel production deploy
api-ci	Pull request	Lint, typecheck, unit and integration tests (PostgreSQL and Redis services), OpenAPI diff check
api-deploy	`main` → staging; tag → production (approval)	Build image, run migrations, deploy, smoke tests

10.9 Go-live checklist

Area	Check
Product	Open decisions in section 13 that block launch are decided and configured.
Demo removal	Everything in appendix B is removed or replaced; no sample data or watermarked images ship.
Legal	Final Terms, privacy policy, cancellation and refund policies, imprint details (company address, managing director, register number, VAT id).
Payments	Stripe live mode, webhooks verified, Connect onboarding tested, refund and payout tested with real small amounts.
Security	External penetration test or security review; staff accounts with two-step verification; secrets rotated from staging.
Operations	Monitoring and alerts on; on-call rota; backup restore tested; support inbox and admin reviewers staffed (24 h application review target).
Stores	Listings, screenshots, privacy forms, reviewer account, age rating; staged rollout plan.
Quality	All acceptance criteria in section 12 pass on Android, iOS and web.

11. Security, privacy and compliance

Note. This section lists the technical measures and the topics that need legal review. It is not legal advice; the company's counsel should confirm each legal point for Germany and the EU.

11.1 Security measures

Area	Measure
Transport	TLS everywhere (HSTS on the web). No plain HTTP endpoints.
Authentication	Argon2id password hashes; hashed one-time codes with expiry and attempt limits; refresh token rotation with reuse detection; per-device sessions; staff accounts require two-step verification.
Authorization	Every endpoint checks role, approval and ownership on the server. Automated tests for access to other users' bookings, tickets, messages and admin endpoints (IDOR).
Input	Schema validation on every request; parameterised queries (ORM); size limits; file type checks on uploads; no user HTML.
Payments	Card entry only through Stripe's UI (keeps PCI DSS scope minimal); webhook signatures verified; idempotency keys; no card data in logs.
Tickets	Ticket codes are random and unguessable (or signed); every scan is validated on the server and logged. DECISION Offline scanning (blueprint F22).
Secrets	Stored in the cloud secrets manager and CI secrets; never in the repository or in `EXPO_PUBLIC_` variables; rotated on staff changes.
Abuse	Rate limits on sign-in, codes, messages, friend requests, reports and search; spam and bot detection on registration.
Data	Encryption at rest for the database, backups and storage; least-privilege access; production data never copied to development.
Supply chain	Dependabot (or similar) for app and API dependencies; lockfiles committed; CI fails on known critical vulnerabilities.
Audit	Append-only audit log for staff decisions and money movements.

11.2 Privacy (GDPR)

- **Hosting and processors:** personal data stays in the EU; list every processor (cloud, Stripe, email, SMS, maps, Sentry, analytics, Expo, Apple, Google) in the records of processing with a DPA.
- **User rights in the app:** data export and account deletion already exist in Account & security; the backend must implement them (export as a downloadable file; deletion with removal of profile, posts, comments, photos, friends and messages).
- **Retention** (as stated in the app's privacy summary): profile and social content until account deletion; booking, payment, refund, payout and ticket records for the period German tax and commercial law requires, also after deletion; support cases and moderation decisions under a retention policy so decisions stay traceable.
- **Location:** used only while the user uses Nearby or the map, and only with permission. Do not store precise location history.

- **Photos:** strip location metadata on upload.
- **Consent:** analytics and any non-essential cookies or device storage need prior consent (web banner, in-app setting).
- **Children and age:** DECISION Minimum age for accounts. Events can carry an age limit (for example 18+); the door team checks ID.

11.3 Legal texts and platform rules to review

Topic	Status and action
Imprint (Impressum)	Screen exists with placeholders: street and number, postcode, managing director, commercial register number, VAT id. Fill in before launch.
Terms and privacy policy	The app shows a plain-language summary labelled as a beta text. Counsel writes the binding texts; the app shows them (or links to the web versions).
Cancellation, refund and no-show policies	DECISION Needed for bookings, tickets and contributions; shown on each booking and event.
Marketplace obligations	Review EU platform rules that apply to a marketplace for services: ranking and "Sponsored" labelling (the app labels promoted events), notice-and-action for reported content with reasons and appeals (the report and appeal flows exist), and reporting of seller income to tax authorities.
Accessibility	Target WCAG 2.1 AA on all platforms; check whether the European Accessibility Act applies to ticket and booking sales.
Image and map rights	19 sample photos carry an "Unsplash+" watermark and `map_berlin.jpg` is a Google Maps screenshot; replace them (appendix B).

12. Testing and acceptance

12.1 Test strategy

Level	Tools (suggested)	Scope
Unit	Jest / Vitest	State machines, payment state mapping, pricing formulas, scores, deadlines, permissions (port the rules in `mobile/src/lib` to server tests first).
API integration	Test runner + real PostgreSQL and Redis containers	Every endpoint: success, validation, permission and conflict cases; jobs with a controllable clock.
Contract	OpenAPI diff in CI	No breaking change without a version bump.
Payments	Stripe test mode, test cards, webhook replay	Authorize, capture, decline, expiry release, failure and retry, refund, payout, dispute.
End-to-end mobile	Maestro or Detox on Android and iOS	The flows in 12.3 on development builds against staging.
End-to-end web	Playwright	Same flows on the web build; responsive widths.
Load	k6 or similar	Feed, search, event page, ticket checkout at sale start, check-in bursts at doors.
Security	Automated scanning + manual review	Access control (IDOR), auth, uploads, webhooks, rate limits.

12.2 Acceptance criteria (from the blueprint)

Criterion	How to test
A member cannot publish a public event without organizer approval.	API: publish as member, artist, venue and unapproved organizer → refused; approved organizer → allowed.
Application status is visible and does not grant premature access.	Pending, info and rejected applicants get `ROLE_NOT_APPROVED` on professional endpoints.
Every provider booking has its own deadline and financial record.	Event with artist and venue bookings: decline one, the other stays unchanged.
Reminder after 12 h only while awaiting a response.	Clock-controlled job test: answered at 11 h → no reminder.
Expiry at 24 h without claiming the release is complete.	Status `expired`, payment `release_pending` until the webhook.
Acceptance alone does not confirm a booking if payment fails.	Accept + failed capture → `payment_failed`, not confirmed.
Rejecting one request does not cancel unrelated confirmed bookings.	As above, plus event cancellation flags bookings instead of cancelling them.
Agreed terms remain after profile prices change.	Change provider fee after booking → booking charges unchanged.
Ticket validation rejects wrong-event, cancelled, refunded and used tickets.	Scan each case; used ticket shows the first check-in time.
Dashboard actions respect ownership and role permissions.	Access other users' resources → 403/404.

Criterion	How to test
Financial totals separate booking value, fees, refunds and payouts.	Ledger totals match Stripe test data.
Empty states show real absence of activity.	New account: every list shows its empty state, no sample content.
All flows work in English on every platform.	E2E suite passes on Android, iOS and web.

12.3 Flow coverage

Flow	Name	Main screens
F01	Member registration	onboarding, register, verify, profile-setup
F02–F03	Professional application and review	apply, application, admin/applications
F04	Sign-in, recovery, role switching	login, forgot-password, workspace, account-security
F05–F07	Artist and venue setup; update profile or availability	pro/services, pro/venue-listing, pro/availability, profile-setup
F08	Discover and evaluate a provider	artists, artist/[id], ratings, venues, venue/[id], map, search
F09–F10	Communities; friends and messages	communities, community/[id], friends, inbox, chat
F11–F13	Private and public events; change or cancel	create-event, event-created, my-events, event-workspace, pro/events
F14–F16	Listed booking, quotes, provider response	booking/new, booking/[id], quote/[id], pro/requests, pro/request/[id]
F17–F19	Failed payment, refund and cancellation, earnings and payout	checkout, booking/[id], booking/issue, pro/earnings
F20–F22	Guests and cost sharing, ticket wallet, check-in	invite, invitation/[id], my-bookings (tickets), ticket/[id], pro/scan, pro/attendees
F23	Discover and book a lesson	lessons, lesson/[id], booking/new
F24–F27	Investigation, moderation, roles, support and appeal	admin/*, support/*, help
F28–F30	Technicians and equipment, user communities, artist projects	LATER not in V1

13. Open decisions

From blueprint section 16. Each decision needs an owner and a date. Until it is made, the teams build the interim behaviour and keep the setting configurable.

Decision	Needed before	Flows	Interim behaviour in this handbook
Authorization and capture vs. charge after acceptance; payment provider; legal payment structure	Payment build	F14–F19	Stripe with both models configurable (section 8.2)
Quote validity, deadline, and whether acceptance replaces provider confirmation	Quote build	F15–F16	Provider sets a validity date; customer acceptance starts payment
Cancellation, refund, no-show, rescheduling and dispute policies	Launch	F13, F18, F20–F22	Policy text per event; refunds only by authorized decision
Who pays commission, final rate, tax presentation, payment fee allocation	Launch	F14–F19	10% commission from provider, 5% customer service fee, configurable
Payout timing, required verification, who receives each charge	Launch	F19–F20	Payout after completion and a configurable hold; Connect verification required
Private cost-sharing limits, host settlement, capacity, re-entry, offline scanning	Ticket build	F20–F22	Online scanning only; one entry per ticket
Public ticket selling, settlement and fee scope	Public event commerce	F12, F20–F22	Organizer ticket sales through the platform, settlement after the event
Sign-in methods, age policy, multiple roles, delegated staff permissions	Access build	F01–F04, F22	Email + password, multiple approved roles, delegated scanner staff per event
Lesson format, recurring sessions, response and cancellation rules	Lesson build	F23	Single sessions; same 24 h response window as bookings
Measurable launch targets and pilot duration	Pilot	Dashboards	Track the dashboard metrics from day one
Reviewer, support and finance authority; escalation policy	Administration	F03, F24–F27	Separate staff permissions; finance approves refunds
Exact venue address visibility before booking	Venue build	F06–F07	Area and approximate pin public; exact address after confirmation
Video posts and live streaming in V1	Media build	Social	Photos only; video and live behind a feature flag
Rental rules, community creation, artist support model	Later phases	F28–F30	Not built in V1

14. Suggested work plan

The phases build on each other. Durations depend on team size and should be estimated by the teams after reading this handbook. Each phase ends with its features working end-to-end on staging on all three platforms.

Phase	Backend	Frontend	Exit criteria
0. Setup	Accounts (section 9), cloud environments, CI/CD, API skeleton, database and migrations, OpenAPI pipeline, seed script	EAS project, development builds, API client generation, env configuration, Sentry	Hello-world deploy to staging; app talks to the staging API
1. Identity and access	Auth, sessions, codes, profiles, preferences, applications, staff permissions, media upload, audit log	Real sign-up, sign-in, reset, account security; profile and avatar upload; application flow; workspace switch	F01–F04 pass; admin can review applications
2. Content and discovery	Events (read and create), venues, artist services, lessons, availability, search, geo, feed, posts, comments with photos, follows, friends, communities, wishlist	Replace mock reads; real map and location; image picker; post and comment creation; empty, loading and error states	F05–F13, F23 (browsing) pass
3. Messaging and notifications	Conversations, WebSocket gateway, notifications, action queue, push and email sending	Realtime chat, push registration and deep links, notification list, attention banner	Messages and notices arrive on all platforms
4. Transactions	Bookings, quotes, changes, cancellations, timers, Stripe payments and webhooks, ticket orders, invitations and contributions, check-in, ledger, Connect payouts, subscriptions	Booking and quote flows, PaymentSheet / Payment Element, ticket wallet, scanner online validation, earnings, Pro plan purchase	F14–F22 pass in Stripe test mode
5. Operations	Support cases, reports and moderation, admin queues and metrics, finance queue, GDPR export and deletion, retention jobs	Support and admin screens on real data; remove demo controls	F24–F27 pass; appendix B cleared
6. Launch	Load and security tests, monitoring and alerts, backup restore test, live keys	Store listings, TestFlight and Play testing, accessibility pass, final legal texts in the app	Go-live checklist (10.9) complete

Appendix A. Glossary

Term	Meaning
Member	Every account's default workspace (internal role `fan`).
Workspace	The active role of an account: Member, Artist, Venue or Organizer. Only approved roles can be active.
Provider	An artist, band, DJ, venue or lesson teacher who receives booking requests.
Host	The account that created an event (private or public). For public events the host is an approved organizer.
Booking	One agreement between a customer and one provider, with its own deadline, price lines and payment state.
Request	A booking seen from the provider's side.
Quote	An itemised, versioned price offer from a provider, valid until a date.
Cost sharing	A private event setting where each accepted guest pays a contribution to receive a ticket.
Action queue	The list of items that need the user's decision (attention banner and dashboard).
Would book again	The musician score: share of verified reviews that recommend booking the artist again.
EAS	Expo Application Services: cloud builds, store submission and over-the-air updates.
Connect	Stripe's marketplace product for paying out providers.

Appendix B. Demo-only items to remove or replace

Item	Where	Replace with
Sample data	`src/data/mock.ts` and initial store state in `src/store/useApp.ts`	API data; seed script for staging only
Device-only persistence of all data	zustand `persist` key `domtak-app`	Server state; persist session, preferences and drafts only
Fixed demo date	`TODAY` in `src/lib/format.ts`	Real current time (server time for rules)
Demo controls (DemoBar)	10 screens: application, booking/[id], booking/issue, event-workspace/[id], help, invitation/[id], pro/request/[id], pro/scan, settings, support/[id]	Remove; real events come from the server
Admin demo switch	`adminMode` toggle in settings and help	Staff permissions from the server
Sample application shortcuts	`SAMPLE_APPLICATION`, `DEMO_MISSING`, `DEMO_REJECT_REASON` in `src/lib/access.ts`	Real applications and admin decisions
Scanner sample codes	`src/app/pro/scan.tsx`	Real camera scans only
Bundled photos referenced by key	`src/data/images.ts`, `assets/images`	CDN URLs from the media service; keep brand art
Watermarked Unsplash+ photos (19)	`av_ben`, `av_sara`, `band_jam`, `dj_kai`, `dj_killers`, `dj_radio`, `dj_red`, `ev_drummer`, `ev_hands`, `ev_party`, `ev_selfie`, `ev_stage_gold`, `feed_crowd`, `feed_drums`, `m_pianist`, `m_sameul`, `m_singer`, `ven_kids`, `ven_neon`	Licensed or own photos (marketing screenshots, onboarding)
Static map image	`map_berlin.jpg` (Google Maps screenshot) and percentage pin positions (`mapPos`)	Map SDK with real coordinates
Simulated payments and payment methods	checkout, payment-methods, `completeCheckout`	Stripe
Placeholder legal details	`src/app/legal.tsx` imprint fields in brackets	Company details and final legal texts

Appendix C. Quick reference

Repository	`github.com/Nasseralmagout/DOMTAK-V1` (app in `mobile/`)
App identifiers	Android `de.domtak.app` · iOS `de.domtak.app` · scheme `domtak://`
Expo SDK	57 (React Native 0.86, React 19.2, TypeScript 6)
Domain model	`mobile/src/data/types.ts`
Business rules	`mobile/src/lib/*.ts` (booking, pricing, ratings, events, access, support, admin)
Store actions	`mobile/src/store/useApp.ts` (about 90 actions mapped in section 7)
Response window	24 h, reminder at 12 h
Demo rates	Service fee 5% · commission 10%
Pro plan	€9,99 / month · €89 / year · first month free
Design reference	Design canvas “Domtak Android App”, page “V1 · Wireframe & final” (136 screens)

End of document.